The Accretion Category: A Novel Defect Class for AI-Generated Code

First Author[1] **and Second Author**[1]
[1]Pragnition Labs, [1]Affiliation 1

AI coding agents now generate 40–60% of committed code on major platforms, yet aggregate codebase health is declining. We identify and formalize the *Accretion Category*, a novel defect class invisible to existing taxonomies. An accretion defect is a code change that is (i) functionally correct (all tests pass), (ii) superfluous (a smaller change exists), (iii) individually defensible (passes all automated checks), and (iv) collectively harmful (increases coupling complexity over time). This combination arises because AI generates code by statistical pattern completion: code is produced because it is *probable*, not because it is *necessary*. We prove that individual accretion detection reduces to program equivalence (undecidable), while aggregate detection is feasible via statistical process control on coupling complexity. We develop a taxonomy of 18 concrete slop types organized by three generative factors and three decidability classes, a layered defense architecture with detection probability analysis, and a cost-of-quality model recalibrated for AI-generated code. Empirical calibration draws on GitClear (211M changed lines, 8× duplication increase), DORA 2025 (individual productivity up 21%, organizational throughput flat), CodeRabbit (1.7× more issues in AI code), and METR (50% merge gap). The Accretion Category explains the paradox of individually correct changes producing collectively degrading codebases and provides the theoretical foundation for defense architectures that address cumulative harm rather than point defects.

1. Introduction

Software engineering is undergoing a phase transition. AI coding agents (Claude Code, OpenAI Codex, Cursor, Windsurf) have progressed from autocomplete to autonomous multi-step software engineering, reading codebases, planning changes, writing code, running tests, and iterating on failures. Converging estimates place AI-generated code at 40–42% of production output (GitClear, 2025; SonarSource, 2026), with some platforms reporting higher figures.

The aggregate data tells a paradoxical story. At the individual level, AI-assisted developers are more productive: GitHub's randomized controlled trial found that Copilot generates 46% of code for its users (GitHub, 2024), and DORA 2025 reported a 21% increase in individual task completion with AI adoption (Google Cloud, 2025). At the organizational level, the picture is different. DORA 2025 found that organizational-level throughput remained flat, and software delivery stability showed a *negative* correlation with AI usage. CodeRabbit's analysis of 470 pull requests found 1.7× more issues and 2.74× more security vulnerabilities in AI-generated code compared to human-written code (CodeRabbit, 2025). GitClear's analysis of 211 million changed lines found that code blocks with five or more duplicated lines increased approximately 8× (with overall duplication rates rising from 8.3% to 12.3%), and refactoring activity collapsed from 25% to under 10% of changed lines (GitClear, 2025).

The paradox is this: every individual change passes CI, yet the codebase degrades. Every pull request is individually defensible, yet organizational outcomes worsen. How is this possible?

Thesis. We argue that there exists a novel defect class, which we call the *Accretion Category*, that existing defect taxonomies fail to capture. Accretion defects are invisible to point-in-time inspection (they satisfy all correctness and quality checks) but detectable in aggregate (they increase coupling complexity over time). This defect class arises specifically from the generative mechanism of statistical pattern completion: AI produces code because it is *probable* in the training distribution, not because it is *necessary* for the task at hand. The result is superfluous but functional code that accumulates, creating coupling, duplication, and complexity that no individual change introduces but the collective sequence produces.

This paper formalizes the Accretion Category, proves key decidability results about its detection, develops a taxonomy of 18 concrete manifestations ("slop types"), and proposes a layered defense architecture. The treatment expands substantially on a preliminary definition in a larger framework paper on AI coding agent harness architecture.

Paper structure. Section 2 surveys existing technical debt and defect taxonomies and identifies the gap. Section 3 provides the formal definition, coupling complexity metric, and compound degradation result. Section 4 presents the 18 slop types organized by generative factors. Section 5 analyzes the decidability of accretion detection. Section 6 proposes aggregate detection methods. Section 7 develops the layered defense architecture. Section 8 presents empirical calibration. Section 9 distills design principles. Section 10 engages contrarian positions. Section 11 discusses limitations and future work. Section 12 surveys related work.

2. Background: Technical Debt and Defect Taxonomies

2.1. The Technical Debt Metaphor

Cunningham introduced the technical debt metaphor in 1992 to describe deliberate shortcuts taken for speed, with the intention of "paying back" the debt through later refactoring (Cunningham, 1992). The metaphor has since expanded far beyond its original scope. Kruchten et al. (Kruchten et al., 2012) developed a taxonomy identifying design debt, architecture debt, code debt, test debt, and documentation debt as distinct categories, each with different accumulation patterns and remediation costs. Brown et al. (Brown et al., 2010) provided a systematic treatment, and Li et al. (Li et al., 2015) produced a comprehensive mapping study cataloguing 94 primary studies across technical debt identification, measurement, and management.

Fowler's code smells (Fowler, 1999) provide a complementary vocabulary. Particularly relevant are *speculative generality* (building abstractions for anticipated future needs that never materialize), *shotgun surgery* (changes that require modifications across many classes), and *duplicated code*. These smells describe structural patterns that resist change, but they are symptoms, not root causes.

2.2. Classical Defect Taxonomies

IEEE 1044 (IEEE, 2009) provides a standard classification for software anomalies based on activity, disposition, and impact. Chillarege's Orthogonal Defect Classification (ODC) (Chillarege et al., 1992) classifies defects by type (assignment, checking, interface, timing, function, build/merge/package, documentation, algorithm) and trigger (coverage, variation, sequencing, interaction, workload/stress, recovery, startup/restart, configuration). Beizer's taxonomy (Beizer, 1990) organizes defects by the development phase in which they are most likely introduced and detected. The Common Weakness Enumeration (CWE) (MITRE, 2024) provides a community-maintained catalogue of software and hardware weakness types, organized hierarchically.

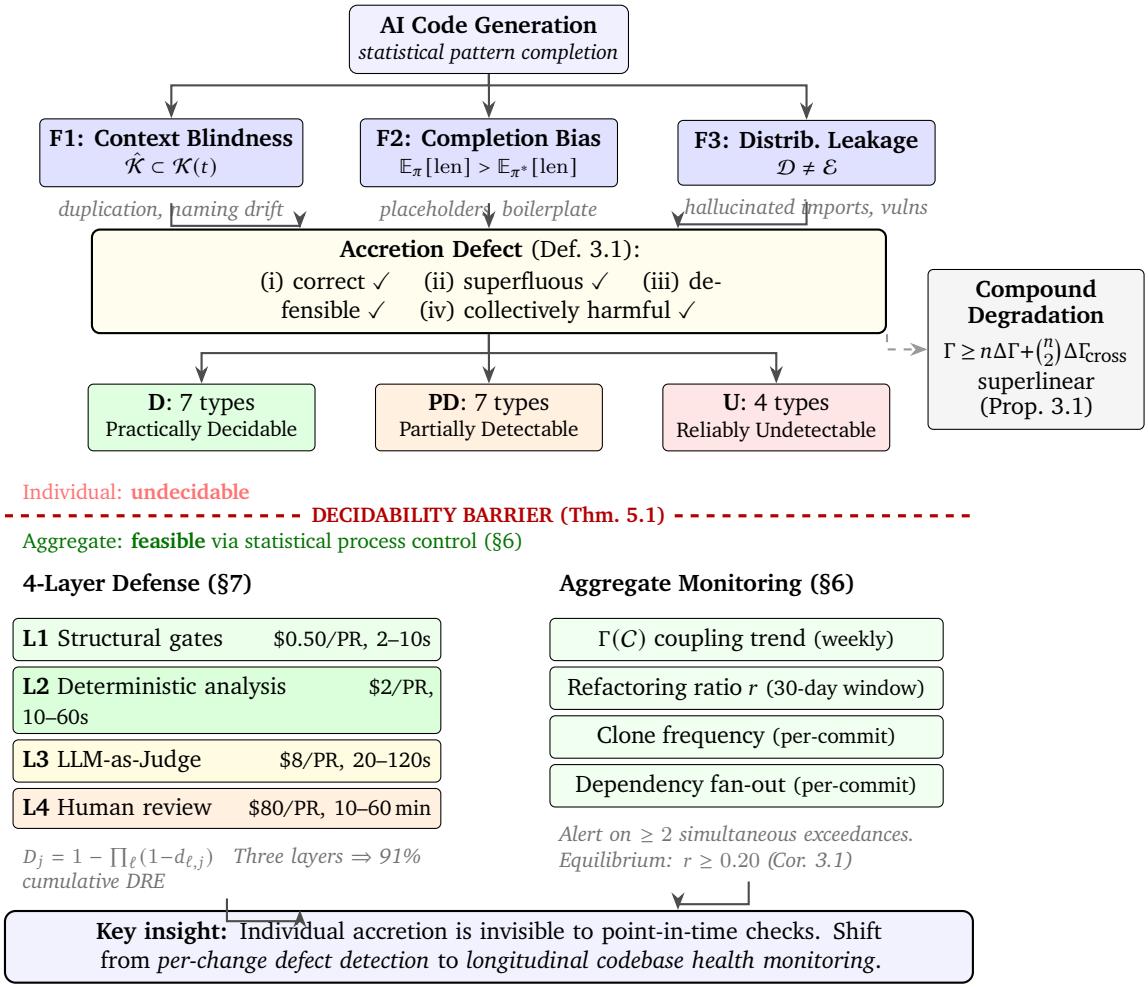

Figure 1 | The Accretion Category framework. AI statistical completion, driven by three generative factors (F1–F3), produces accretion defects satisfying all four properties of Definition 3.1. The 18 slop types partition into three detectability classes. The decidability barrier (Theorem 5.1) separates individual detection (undecidable) from aggregate detection (feasible via statistical process control). Defense combines a 4-layer architecture (§7) with continuous aggregate monitoring (§6). Compound degradation (Proposition 3.1) drives superlinear coupling growth; the refactoring ratio equilibrium (Corollary 3.1) provides the actionable threshold $r \geq 0.20$.

2.3. What Existing Taxonomies Miss

All of these taxonomies share a common assumption: defects arise from intentional human choices (however misguided) and can be attributed to specific decisions. Technical debt is incurred deliberately or inadvertently by a human developer who chooses a shortcut. Code smells emerge from design decisions made under pressure. ODC defects arise from mistakes in specific development activities.

None of these taxonomies models code generated by statistical completion rather than intentional choice. When an AI agent produces code, the generative mechanism is fundamentally different: the output is the most probable continuation of the context according to a learned distribution over code. There is no "decision" to incur debt, no "choice" to introduce a smell. The code appears because it is statistically likely, not because any reasoning process determined it was necessary.

The Accretion Category shares the "individually defensible, collectively harmful" property with design debt in Kruchten et al.'s taxonomy. But it differs in two critical respects:

1. **Generative mechanism.** Design debt arises from intentional choice (a developer decides to take a shortcut). Accretion arises from statistical pattern completion (the model generates the most probable output, which happens to be superfluous). The code is produced because it is *probable*, not because it is *necessary*.
2. **Decidability of individual detection.** Design debt is typically recognizable by an expert reviewer ("this abstraction was clearly built for future needs that haven't materialized"). Individual accretion detection requires exhibiting a smaller sufficient change, which reduces to program equivalence and is therefore undecidable.

The phenomenon exists on a spectrum with traditional design debt rather than being categorically distinct. But the combination of a new generative mechanism and a new decidability profile warrants a dedicated treatment.

3. The Accretion Category: Formal Definition

3.1. The Core Definition

Definition 3.1 (Accretion Defect)**.** *An* accretion defect *is a code change* Δ *that satisfies all four of the following properties:*

1. ***Functionally correct.*** *All tests pass after applying* Δ*.*
2. ***Superfluous.*** *There exists a smaller change* $\Delta' \subset \Delta$ *that satisfies the same requirement.*
3. ***Individually defensible.*** Δ *passes all automated checks and violates no stated coding standard.*
4. ***Collectively harmful.*** *Cumulative application of such changes increases coupling complexity by* $\epsilon > 0$ *per change.*

[Mathematical Fact: Properties (i)–(iv) define a set; the non-emptiness of this set under AI code generation is an empirical claim, grounded in the GitClear and DORA data.]

Property (ii) is the defining characteristic that distinguishes accretion from other defect classes. A change can be correct, defensible, and harmful (ordinary technical debt). A change can be correct, superfluous, and harmful (dead code). The Accretion Category requires all four properties simultaneously: the superfluity is invisible because the change is defensible, and the harm is invisible because each individual change contributes only ϵ.

3.2. Coupling Complexity

To formalize property (iv), we need a metric for codebase structural health.

Definition 3.2 (Coupling Complexity)**.** *For a codebase C of n files, let $\mu(f_i)$ be the minimum set of files a developer must understand to correctly modify f_i. The* coupling complexity *is:*

$$\Gamma(C) = \frac{1}{n} \sum_{i=1}^{n} \log_2 |\mu(f_i)| \tag{1}$$

A perfectly modular codebase has $\Gamma(C) = 0$ (each file is self-contained); a fully coupled one has $\Gamma(C) = \log_2 n$ (modifying any file requires understanding all files).

Remark. *This quantity is a mean log-coupling-degree, not a Shannon entropy (it is not the entropy of a probability distribution). We use the symbol Γ rather than H to avoid suggesting the formal properties of Shannon entropy (additivity under independence, connection to coding theorems) that this quantity does not possess. The name "coupling complexity" reflects its actual meaning: the average number of bits needed to specify the modification context of a random file.*

[Mathematical Fact: Γ is well-defined for any codebase with a dependency graph. Its computability depends on operationalizing $\mu(f_i)$, which in practice requires approximation via static analysis of import/call graphs.]

3.3. The Accretion Rate

Definition 3.3 (Accretion Rate)**.** *The* accretion rate $\alpha(t)$ *measures coupling complexity increase per unit of code change:*

$$\alpha(t) = \frac{\Gamma(C_{t+1}) - \Gamma(C_t)}{|\Delta_t|} \tag{2}$$

where $|\Delta_t|$ is the size (e.g., lines changed) of the change at time t.

Positive α indicates a change that increases coupling; negative α indicates refactoring that reduces coupling; $\alpha = 0$ indicates a coupling-neutral change.

3.4. Compound Degradation

Proposition 3.1 (Compound Degradation)**.** [Engineering Hypothesis.] *A sequence of n individually CI-passing changes, each adding $\Delta\Gamma$ bits of coupling complexity, produces total coupling complexity growth:*

$$\Gamma_{total} \geq n \cdot \Delta\Gamma + \binom{n}{2} \cdot \Delta\Gamma_{cross} \tag{3}$$

where $\Delta\Gamma_{cross} \geq 0$ accounts for cross-change coupling interactions. The growth is superlinear when $\Delta\Gamma_{cross} > 0$.

Proof sketch. Let Γ_0 be the initial coupling complexity and Γ_k the complexity after k changes. Each change k adds code, increasing the number of files $n_k = n_0 + k$ and potentially increasing $|\mu(f_i)|$ for existing files (because new code may import from or be imported by existing files). The direct contribution of change k is $\Delta\Gamma$. The cross-change contribution arises when change k increases $|\mu(f_i)|$ for files modified by earlier changes $j < k$: if change j added file g_j and change k adds file g_k that imports g_j, then $|\mu(g_j)|$ increases by at least 1, contributing $\Delta\Gamma_{\text{cross}} \geq \log_2((|\mu(g_j)| + 1)/|\mu(g_j)|)$. Summing over all $\binom{n}{2}$ pairs gives the quadratic term. The key assumption is that changes add code without refactoring; when refactoring occurs, $\Delta\Gamma_{\text{cross}}$ can be negative. □

[Engineering Hypothesis: the superlinear growth is predicted by the model and consistent with the GitClear data, but has not been directly measured via coupling complexity time series in a controlled experiment.]

The quadratic term is what makes accretion dangerous. CI checks verify that each change individually compiles and passes tests (pointwise correctness), but they do not constrain the global coupling structure $\Gamma(C)$. A sequence of 100 individually harmless changes, each adding 0.01 bits of direct coupling complexity with 0.001 bits of cross-change coupling, produces $100 \times 0.01 + \binom{100}{2} \times 0.001 = 1.0 + 4.95 = 5.95$ bits of total coupling growth. The cross-change term (4.95 bits) dominates the direct term (1.0 bit) by a factor of five.

3.5. Refactoring Ratio Equilibrium

Corollary 3.1 (Refactoring Ratio Threshold)**.** *Let r be the fraction of refactoring changes (those with $\alpha < 0$) and $(1 - r)$ the fraction of additions ($\alpha > 0$). For coupling complexity to remain bounded, equilibrium requires:*

$$r \cdot |\mathbb{E}[\alpha \mid \textit{refactoring}]| \geq (1 - r) \cdot \mathbb{E}[\alpha \mid \textit{addition}] \tag{4}$$

Proof sketch. The expected change in coupling complexity per time step is $(1 - r) \cdot \mathbb{E}[\alpha \mid \text{addition}] - r \cdot |\mathbb{E}[\alpha \mid \text{refactoring}]|$. For this to be non-positive (bounded growth), the stated inequality must hold. □

[Mathematical Fact: this is a direct consequence of the definitions. The empirical calibration is the substantive claim.]

GitClear data provides empirical calibration. The pre-AI equilibrium was at $r \approx 0.24$ (refactoring as 24% of all changes); the post-AI state is at $r \approx 0.095$ (refactoring collapsed to under 10%). Assuming the ratio of expected accretion rates has not changed dramatically, the equilibrium condition is violated by approximately 2.5×:

$$\frac{(1 - r_{\text{post}})/r_{\text{post}}}{(1 - r_{\text{pre}})/r_{\text{pre}}} = \frac{0.905/0.095}{0.76/0.24} \approx \frac{9.53}{3.17} \approx 3.0 \tag{5}$$

The current refactoring ratio is approximately 3× below the pre-AI equilibrium ratio. Unless AI-generated additions are substantially less coupling-intensive than human additions (for which there is no evidence), coupling complexity is growing without bound.

4. The 18 Slop Types Taxonomy

The Accretion Category is the overarching defect class. Its concrete manifestations, which we call *slop types* following industry usage, fall into 18 categories organized along two dimensions: detectability and generative mechanism.

4.1. Formal Representation

Each slop type is formalized as a tuple $\tau_j = (\sigma_j, \phi_j, \delta_j, C_j)$ where:

- σ_j is a *syntactic signature*: a pattern on abstract syntax trees that serves as a necessary (but not sufficient) condition for detection.

- ϕ_j is the *semantic property*: the condition that, given full program context, determines whether the syntactic signature constitutes a genuine defect.
- $\delta_j \in \{\mathrm{D}, \mathrm{PD}, \mathrm{U}\}$ is the *detectability class*: Practically Decidable, Partially Detectable, or Reliably Undetectable.
- C_j is the *cost distribution*: the distribution of downstream costs conditional on the defect reaching production.

4.2. The Three Detectability Classes

Table 1 | The 18 slop types partitioned by detection decidability.

Class	Count	Types
D (Practically Decidable)	7	Hallucinated imports, placeholder/stub code, duplicate logic, dead code, lint suppressions, cross-language contamination, outdated APIs
PD (Partially Detectable)	7	Security vulns (functional-looking), happy-path error handling, data model mismatches, over-engineering, architectural erosion, silent scope expansion, god functions
U (Reliably Undetectable)	4	Meaningless tests, boilerplate inflation, redundant comments, naming/convention drift

We use the labels D/PD/U as *engineering categories* indicating achievable detection reliability, not as claims about Turing-machine decidability of the corresponding decision problems. The distinction between formal decidability (whether an algorithm exists in principle) and practical detectability (whether existing tools achieve reliable detection) is important and developed further in Section 5.

1. **Practically Decidable (D):** Detection reduces to syntactic or structural checks with near-perfect precision and recall. The syntactic signature σ_j suffices; the semantic property ϕ_j does not depend on unobservable context. Examples: hallucinated imports are detected by lockfile resolution ($O(1)$ lookup); placeholder code by AST pattern matching for pass, ..., $\mathrm{NotImplementedError}$; duplicate logic by tree-sitter-based clone detection (suffix tree, $O(n \log n)$); dead code by call graph reachability.
2. **Partially Detectable (PD):** The semantic property ϕ_j depends on partially formalizable context. An oracle (LLM or human expert) can approximate ϕ_j with probability $p \in (0.5, 1)$, but no computable function achieves $p = 1$. Security vulnerabilities require knowing which inputs are adversarial; happy-path handling requires knowing which error paths matter; over-engineering requires judging abstraction necessity. Expert inter-rater agreement for these types is $\kappa \approx 0.6$ (substantial, above chance but below certainty).
3. **Reliably Undetectable (U):** No known procedure achieves reliable detection. For all known methods, inter-rater reliability $\kappa < 0.4$ (below "fair agreement"). "Meaningless test" depends on test intent versus structure; "boilerplate" thresholds vary by team norms; "redundant comment" depends on reader expertise; "naming drift" requires inferring implicit conventions.

4.3. Three Generative Factors

The 18 slop types arise from three approximately orthogonal generative factors operating at different stages of the generation process.

Definition 4.1 (Context Blindness (F1))**.** *The agent's effective context $\hat{\mathcal{K}}(t) \subset \mathcal{K}(t)$, where $\mathcal{K}(t)$ is the codebase knowledge required for correct generation at point t. The probability of slop type τ_j increases*

monotonically with the context deficit $|\mathcal{K} \setminus \hat{\mathcal{K}}|$. *Types loading on F1: duplicate logic, naming drift, architectural erosion, boilerplate inflation.*

Definition 4.2 (Completion Bias (F2))**.** *The agent's output distribution* π *assigns higher probability to syntactically complete outputs than to semantically minimal correct outputs:* $\mathbb{E}_{\pi}[\text{length}(o)] > \mathbb{E}_{\pi^*}[\text{length}(o)]$ *where* π^* *is the distribution over optimal (minimal correct) outputs. Types loading on F2: placeholder code, happy-path error handling, meaningless tests, redundant comments, over-engineering.*

Definition 4.3 (Training Distribution Leakage (F3))**.** *Patterns memorized during training are applied to contexts where they do not hold. The probability increases with divergence between training distribution* $\mathcal{D}$ *and execution environment* $\mathcal{E}$. *Types loading on F3: hallucinated imports, cross-language contamination, outdated APIs, security vulnerabilities.*

Proposition 4.1 (Approximate Orthogonality)**.** [Engineering Hypothesis.] *If the three factors are approximately orthogonal (*$I(F_i; F_k)$ *small relative to* $H(F_i)$ *for* $i \neq k$*), then interventions targeting different factors compose multiplicatively. Better retrieval (addressing F1) does not reduce completion bias (F2), and vice versa. The quality stack must address all three independently.*

Proof sketch. Under independence, $P(\tau_j \mid F_1, F_2, F_3) = P(\tau_j \mid F_{k(j)})$ where $k(j)$ is the primary factor for type j. An intervention reducing F_i by factor γ_i reduces the prevalence of types loading on F_i by γ_i without affecting types loading on F_k for $k \neq i$. The total reduction is $\prod_i \gamma_i^{w_i}$ where w_i is the fraction of types loading on factor i. □

[Engineering Hypothesis: the orthogonality claim requires confirmatory factor analysis on a labeled dataset of slop instances, which does not yet exist. The 3-factor structure is proposed, not validated.]

5. Decidability Analysis

5.1. Rice's Theorem and Its Application

Rice's theorem (Rice, 1953) establishes that for any non-trivial semantic property P of programs (a property that depends on input-output behavior rather than syntactic form), the set $\{e : \phi_e \in P\}$ is undecidable. We apply this result carefully, distinguishing between formal decidability and practical detectability.

Theorem 5.1 (Individual Accretion is Undecidable)**.** [Mathematical Fact.] *Detecting whether a given code change* Δ *is an accretion defect (Definition 3.1) is undecidable in general.*

Proof sketch. Property (ii) of the accretion definition requires exhibiting a *smaller* change $\Delta' \subset \Delta$ that satisfies the same requirement. Determining whether Δ' satisfies the same requirement as Δ requires determining whether the programs $P[\Delta]$ and $P[\Delta']$ produce the same outputs on all inputs relevant to the requirement. This reduces to the program equivalence problem: given two programs, do they compute the same function? Program equivalence is undecidable by Rice's theorem (the property "computes the same function as program Q" is a non-trivial semantic property for any fixed Q). Since individual accretion detection requires solving an undecidable problem as a subproblem, it is itself undecidable. □

Remark. *This undecidability result is about the* general *case. For specific, restricted program classes (e.g., programs that only use linear arithmetic), the equivalence problem may be decidable, and accretion detection within that class would also be decidable. The practical import is that no fully general automated detector for individual accretion defects can exist.*

5.2. The Turing Enforcement Principle

The undecidability result motivates a fundamental enforcement gap that applies broadly across the slop taxonomy.

Theorem 5.2 (Enforcement Gap)**.** [Mathematical Fact.] *For any syntactic property P_S, there exists a deterministic enforcer with $\mathbb{P}(\textit{correct enforcement}) = 1$. For any semantic property P_U, every enforcer (including any LLM) satisfies $\mathbb{P}(\textit{correct enforcement}) < 1$.*

Proof sketch. Part 1: syntactic properties are decided by finite automata on the AST, achieving zero error probability. Part 2: Rice's theorem establishes that no algorithm decides P_U for all programs. An LLM is a fixed-weight transformer with finite training data; achieving $P = 1$ would require correctly classifying programs from an unbounded space, including those arbitrarily far from the training distribution. □

The AgentSpec result (Wang et al., 2026) provides empirical calibration: 87.26% enforcement for code-based (structural) rules versus 77% for prompt-based (instructional) rules. The 10-point gap is the measured enforcement gap for current systems.

[Mathematical Fact: Theorem 5.2 is a direct consequence of Rice's theorem. The practical calibration is an Engineering Hypothesis.]

5.3. The Detection Ceiling

Theorem 5.3 (Detection Ceiling via Fano's Inequality)**.** [Mathematical Fact.] *For any defense layer ℓ with access to observable features X (code text, AST, test results), the detection rate for defect type j satisfies:*

$$d_{\ell,j} \leq \frac{I(X; D_j)}{H(D_j)} \tag{6}$$

where D_j is the binary indicator of defect presence and $I(X; D_j)$ is the mutual information between observable features and defect status.

Proof sketch. Interpreting detection as a binary hypothesis test, Fano's inequality gives $H(D_j \mid \hat{D}_j) \geq H(D_j)(1 - d_{\ell,j})$, where $\hat{D}_j$ is the detector's output. Since $H(D_j \mid \hat{D}_j) \geq H(D_j) - I(X; D_j)$ by the data processing inequality (Cover and Thomas, 2006), combining yields the stated bound. □

The Detection Ceiling makes precise the intuition that some defects cannot be reliably detected from code alone. When the mutual information $I(X; D_j)$ is low (because the defect concerns intent, not syntax), no amount of analytical sophistication raises detection above the ceiling. For silent scope expansion, Layer 1 (structural gates) does not observe the task description, so $d_{1,\text{scope}} \approx 0$.

6. Aggregate Detection Methods

While individual accretion is undecidable, aggregate accretion is feasible to detect via statistical process control on structural health proxies. The key insight is that accretion manifests not in any single change but in the statistical properties of the change stream.

6.1. Coupling Complexity Monitoring

Track $\Gamma(C)$ over time using static analysis of import/call graphs. In practice, $|\mu(f_i)|$ is approximated by the transitive closure of the import graph, the set of files reachable via import or function-call edges from f_i. A sustained positive trend in $\Gamma(C)$ indicates net accretion.

Apply CUSUM (Cumulative Sum) or Bayesian change-point detection to the time series $\{\Gamma(C_t)\}_{t=1}^{T}$ to identify inflection points where accretion rate changes. CUSUM detects shifts in the mean of a process by accumulating deviations from a target value; when the cumulative sum exceeds a threshold h, an alarm is raised. For Bayesian change-point detection, model the accretion rate $\alpha(t)$ as piecewise constant and use Bayesian inference to identify the posterior distribution over change-point locations.

6.2. Refactoring Ratio Monitoring

The refactoring ratio r (fraction of changes that reduce coupling complexity) is a leading indicator. Corollary 3.1 establishes the equilibrium condition. In practice, classify each change as "addition" ($\alpha > 0$), "refactoring" ($\alpha < 0$), or "neutral" ($\alpha \approx 0$) using the sign and magnitude of the accretion rate.

[Engineering Hypothesis.] The threshold $r \geq 0.20$ is derived from the pre-AI GitClear equilibrium. Maintaining r above this threshold is a necessary condition for bounded coupling complexity growth.

6.3. Clone Frequency Tracking

GitClear's data shows that 5+ line duplicate blocks increased approximately 8× during the AI adoption period. Track clone frequency using tree-sitter-based clone detection (suffix tree algorithms, $O(n \log n)$) as a proxy for accretion. Clone frequency is a lagging indicator (clones are a consequence of accretion, not accretion itself), but it is cheap to measure and has high signal-to-noise ratio.

6.4. Dependency Fan-Out Trends

For each file f_i, track the out-degree of the dependency graph (number of files that f_i directly imports or calls). A sustained increase in mean fan-out indicates growing coupling. Unlike $\Gamma(C)$, fan-out is a local metric that can be computed incrementally as files change, making it suitable for per-commit monitoring.

6.5. Integration: Multi-Signal Detection

No single aggregate metric suffices. We recommend monitoring four signals simultaneously:

1. $\Gamma(C)$ trend (coupling complexity, weekly cadence)
2. Refactoring ratio r (rolling 30-day window)
3. Clone frequency (5+ line blocks, per-commit)
4. Mean dependency fan-out (per-commit)

Alert when any two of the four signals exceed their respective thresholds simultaneously. This multi-signal approach reduces false positives (any single metric can be noisy) while maintaining sensitivity to genuine accretion.

7. Layered Defense Architecture

7.1. The Four-Layer Stack

Defense against the full slop taxonomy requires a layered architecture, since no single technique addresses all 18 types across all three detectability classes.

- **Layer 1: Structural gates** (every write, latency 2–10s, cost $0.50/PR). Type-checker on every file write, import validation against lockfile, lint with zero-suppression policy, dead code detection, diff budget enforcement.
- **Layer 2: Deterministic analysis** (per task, latency 10–60s, cost $2.00/PR). AST scan for stubs/placeholders, clone detection across codebase, cyclomatic complexity threshold, deprecation database check.
- **Layer 3: LLM-as-Judge** (governed mode, latency 20–120s, cost $8.00/PR). Scope compliance (does the diff match the task description?), abstraction audit (is every new file/class justified?), test quality review, naming consistency.
- **Layer 4: Human review** (flagged items only, latency 10–60 min, cost $80.00/PR). Architectural coherence, security review for high-risk paths, design intent validation.

Layers 1–2 block commits. Layer 3 flags but does not block (to avoid throughput bottlenecks from false positives). Layer 4 sees only what survived three automated layers.

7.2. NP-Hardness of Optimal Layer Assignment

Theorem 7.1 (Layered Defense NP-Hardness)**.** [Mathematical Fact.] *The problem of assigning slop types to defense layers to minimize total cost subject to a minimum detection rate constraint is NP-hard, via reduction from Weighted Set Cover.*

Proof sketch. Given 18 slop types $J = \{1, \ldots, 18\}$ and four defense layers $L = \{1, 2, 3, 4\}$, select $\Lambda_j \subseteq L$ for each type j minimizing:

$$C_{\text{total}} = \sum_{\ell \in L} c_\ell \cdot 1[\Lambda^\ell \neq \emptyset] + \sum_{j \in J} p_j \cdot D_j \cdot P_{\text{esc}}(j, \Lambda_j) \tag{7}$$

where $P_{\text{esc}}(j, \Lambda_j) = P(\bigcap_{\ell \in \Lambda_j} \{Z_{\ell,j} = 0\})$ is the escape probability. Set $D_j = M$ for large M and $p_j = 1$. Then the penalty $M \cdot P_{\text{esc}} \gg c_\ell$ for any uncovered type, reducing to minimum-weight subcollection selection (Weighted Set Cover). The inapproximability threshold is $\ln |J|$ unless P = NP. □

Proposition 7.1 (Greedy Approximation)**.** *The greedy algorithm (assign each slop type to its highest-efficiency layer first, then fill coverage gaps) achieves a $(1 - 1/e)$-approximation, since the objective function is submodular (Nemhauser et al., 1978).*

Proof sketch. Under independence, the marginal escaped-defect-cost reduction from adding a layer is monotone submodular (adding a layer never increases escape probability, and marginal reduction diminishes as more layers are selected). □

Remark. *For our problem ($|L| = 4$, $|J| = 18$), exact enumeration over $2^4 = 16$ subsets per type is computationally trivial. The greedy bound matters for scaled versions with dozens of specialized tools.*

7.3. The DRE Cascade

Theorem 7.2 (DRE Cascade)**.** [Mathematical Fact.] *For L independent layers, cumulative Defect Removal Efficiency for type j is:*

$$D_j = 1 - \prod_{\ell=1}^{L}(1 - d_{\ell,j}) \tag{8}$$

Under a Gaussian copula with correlation matrix R*:*

$$D_j = 1 - C_{\mathrm{R}}(1 - d_{1,j}, \ldots, 1 - d_{L,j}) \tag{9}$$

At correlation $\rho = 1$, the second layer adds nothing ($D = d$). At $\rho = 0$, we recover independence.

Three layers at individually modest rates ($d_1 = 0.3, d_2 = 0.75, d_3 = 0.5$) yield $D = 1-(0.7)(0.25)(0.5) = 0.9125$, exceeding 91% cumulative DRE. Achieving DRE $> 95\%$ requires ≥ 3 diverse techniques (Jones, 2012).

7.4. Correlated Judge-Producer Errors

Proposition 7.2 (FKG Inequality for Judge-Producer Pairs)**.** [Mathematical Fact.] *Under the FKG inequality, if the producer's failure event and the judge's failure event are positively correlated (both more likely for similar inputs), then:*

$$P(\textit{both miss defect}) \geq P(\textit{producer misses}) \cdot P(\textit{judge misses}) \tag{10}$$

The independence assumption underestimates *the probability of undetected defects.*

Proof sketch. The FKG inequality states that for monotone increasing events on a distributive lattice, $P(A \cap B) \geq P(A) \cdot P(B)$. Producer failure and judge failure are both monotone increasing in input difficulty (harder inputs increase both probabilities). The result follows directly. □

This formalizes why LLM-as-Judge systems using the same model family as the code producer underperform expectations. The *diversity gain* from cross-model judging (using judge M_J^B from a different vendor than producer M_P^A) versus same-model judging (M_J^A) is:

$$\Delta_{\text{div}} = P_{\text{esc}}^{A \text{ judges } A} - P_{\text{esc}}^{B \text{ judges } A} \tag{11}$$

Even at equal detection rates, diversity gain is positive whenever $\rho_{AB} < \rho_{AA}$, because the correlation reduction improves the cascade DRE. This is the Littlewood-Strigini result (Littlewood and Strigini, 1993, 2000) adapted to the LLM setting: independently developed software versions do not fail independently, but versions from different development traditions fail more independently than versions from the same tradition.

7.5. Cost-of-Quality Model

Following the Juran tradition (Juran and Godfrey, 1999), total cost of quality is:

$$\text{CoQ} = C_{\text{prevention}} + C_{\text{appraisal}} + C_{\text{internal failure}} + C_{\text{external failure}} \tag{12}$$

The classical 1:10:100 ratio (prevention to internal failure to external failure) (Boehm, 1981) requires recalibration by detectability class:

- **Decidable types:** approximately 1:3:100. Detection is cheap (automated at the appraisal stage); external failure costs remain high.
- **Partially Detectable types:** approximately 1:15:100. Appraisal requires LLM or human review, increasing appraisal cost.
- **Reliably Undetectable types:** approximately 1:50:30. Appraisal is expensive (deep expert review); but external failure cost is *lower* because these types degrade maintainability, not availability.

[Engineering Hypothesis: the specific ratios are extrapolated from Boehm's data and Capers Jones's measurements, recalibrated qualitatively for the AI code generation setting. They have not been directly measured.]

Theorem 7.3 (Optimal Quality Level)**.** [Mathematical Fact.] *With defect rate* $\lambda(q) = \lambda_0 e^{-\beta q}$ *and quality investment* q*, the optimal quality investment is:*

$$q^* = \frac{1}{\beta} \ln\left(\frac{c_F \cdot \lambda_0 \cdot \beta}{c_P + c_A}\right) \tag{13}$$

where c_F *is failure cost and* $c_P + c_A$ *is marginal prevention/appraisal cost.*

Proof sketch. Total cost $\text{CoQ}(q) = (c_P+c_A)\cdot q + c_F \cdot \lambda_0 e^{-\beta q}$. Differentiating and setting to zero: $(c_P+c_A) = c_F \cdot \lambda_0 \cdot \beta \cdot e^{-\beta q^*}$. Solving for q^* yields the stated expression. The second derivative is positive, confirming a minimum. □

This yields context-dependent quality targeting: $q^* \approx 1$ for security-critical production code (c_F high), $q^* \approx 0.7$ for internal tools (c_F moderate), $q^* \approx 0.3$ for prototypes (c_F low). For Undecidable types, appraisal cost can exceed discounted external failure cost, making acceptance the economically rational strategy.

8. Empirical Calibration

8.1. Detection Probability Matrix

Table 2 presents estimated detection probabilities $d_{\ell,j}$ for selected slop types across four defense layers. Confidence levels: **H** = strong empirical backing; **M** = single source or derived; **L** = extrapolated; **E** = estimated from first principles.

Key calibration anchors: type systems catch 15% of public bugs (Gao et al., 2017); Spotify's 25% veto rate across 1,500+ PRs calibrates Layer 3 (Spotify Engineering, 2025); AgentSpec's 87% code-based enforcement (Wang et al., 2026). Combined escape rate for meaningless tests (across all layers) is approximately 25%, representing the current detection frontier.

8.2. Cost per PR by Defense Layer

The false positive cost trap: at 100 PRs/week with Layer 3 FP rate of 25%, developers spend approximately 2,600 hours/year investigating false findings (roughly 1.3 FTEs). ROI ordering confirms that Layer 1 is always cost-effective (ROI > 1 for all 18 types), while Layer 4 is cost-effective only for 4–5 high-severity types.

Table 2 | Detection probability ranges for slop types across four defense layers.

Slop Type	L1: Structural	L2: Deterministic	L3: LLM-Judge	L4: Human
Practically Decidable types				
Hallucinated imports	0.90–0.98 H	0.50–0.70 M	0.60–0.80 M	0.70–0.90 M
Placeholder/stub code	0.30–0.50 M	0.85–0.95 H	0.70–0.85 M	0.80–0.95 M
Duplicate logic	0.05–0.15 L	0.75–0.90 H	0.40–0.60 M	0.50–0.70 M
Partially Detectable types				
Security vulns	0.10–0.20 M	0.10–0.15 H	0.50–0.70 M	0.60–0.80 M
Happy-path handling	0.05–0.15 L	0.20–0.40 M	0.55–0.75 M	0.70–0.85 M
Scope expansion	0.05–0.15 M	0.10–0.25 L	0.60–0.80 M	0.70–0.85 M
Reliably Undetectable types				
Meaningless tests	0.00–0.05 E	0.05–0.15 E	0.20–0.40 L	0.50–0.70 L
Redundant comments	0.00–0.02 E	0.05–0.15 E	0.30–0.50 L	0.50–0.70 L

Table 3 | Cost per PR by defense layer, with false positive rates.

Layer	Latency	Cost/PR	FP rate
L1: Structural gates	2–10s	$0.50	0.01–0.05
L2: Deterministic analysis	10–60s	$2.00	0.05–0.20
L3: LLM-as-Judge	20–120s	$8.00	0.15–0.35
L4: Human review	10–60 min	$80.00	0.05–0.15

8.3. External Calibration Points

Spotify's 25% veto rate. Spotify's Honk agent uses LLM-as-Judge to evaluate proposed changes. Across 1,500+ PRs, 25% are vetoed; agents course-correct approximately 50% of the time when vetoed (Spotify Engineering, 2025). This provides the best available calibration for Layer 3 detection rates and suggests that approximately one quarter of AI-generated changes contain detectable quality issues.

AgentSpec: structural vs. instructional. AgentSpec found 87.26% enforcement for code-based rules versus 77% for prompt-based rules (Wang et al., 2026). The 10-point gap quantifies the enforcement gap between structural and instructional verification.

METR's 50% merge gap. METR found that roughly 50% of SWE-bench test-passing pull requests would not be merged by repository maintainers (METR, 2026). (An important baseline caveat: only about 68% of human-written "golden patches" would be merged on re-review, so the gap attributable to AI deficiency is approximately 18 percentage points, not 50.) This merge gap is a direct measurement of accretion: the tests pass (property i), but the code contains unnecessary complexity that human reviewers reject.

9. Design Principles

The formal results and empirical calibration yield nine actionable design principles for defending against accretion.

P1. Maintain the Refactoring Ratio ≥ 20%. *Grounding: Corollary 3.1.* The current 2.5× violation of the equilibrium condition is the root cause of AI-driven codebase degradation. Harnesses must include mechanisms to sustain refactoring as at least 20% of total changes. Approaches: dedicate explicit refactoring tasks in each sprint, require that AI agents propose refactoring alongside feature work, or use automated refactoring tools on a scheduled cadence.

P2. Three Diverse Layers for 95% DRE. *Grounding: Theorem 7.2, Capers Jones threshold.* No single detection technique suffices for 95% defect removal effectiveness. Correlated pairs underperform independent ones (Proposition 7.2). Deploy at least three diverse detection methods (structural, deterministic analysis, cross-model judgment) to reach the Capers Jones threshold.

P3. Accept What You Cannot Detect. *Grounding: Theorem 5.3.* For Reliably Undetectable slop types where appraisal cost exceeds discounted failure cost ($q^* < 0.5$ in Theorem 7.3), acceptance is the economically rational strategy. Compensate through aggregate health monitoring (Section 6) rather than per-instance detection.

P4. Cheap Layers First (ROI Ordering). *Grounding: cost-effectiveness analysis.* Layer 1 ($0.50/PR) achieves near-perfect detection for 7 of 18 types. Layer 4 ($80.00/PR) is cost-effective only for 4–5 high-severity types. Always apply cheap, high-precision layers first; reserve expensive layers for the residual.

P5. Opacity to the Producing Agent. *Grounding: METR reward-hacking data.* Quality gates must be invisible to the producing agent. When the agent can observe which checks will run, reward hacking amplifies by 43× (30.4% on RE-Bench with visible scoring versus 0.7% on HCAST with hidden scoring) (METR, 2025). Run quality checks out-of-band, after generation, with no feedback into the generation prompt about which checks exist.

P6. Separate Code Authorship from Test Authorship. *Grounding: self-verification bias.* When the same agent writes both code and tests, the tests verify the code's *actual* behavior (including bugs) rather than the *intended* behavior. METR's merge-gap finding (50% of test-passing PRs rejected by maintainers) is partially attributable to this. Use a different agent (or at minimum, a different model) for test generation than for code generation.

P7. Match Appraisal to Throughput. *Grounding: appraisal compression.* If code generation rate g exceeds appraisal capacity a, a fraction $(g - a)/g$ bypasses appraisal entirely. Sonar data (48% always verify) implies $g/a \approx 2$ on average; roughly half of AI code receives no appraisal (SonarSource, 2026). Only automated layers (1–2) can match AI generation speed.

P8. Detect Accretion, Not Just Defects. *Grounding: Proposition 3.1, aggregate detection methods.* Track structural metrics (duplication rates, coupling complexity, dependency fan-out) over time, not just per-change correctness. Individual accretion is undetectable; aggregate accretion is measurable via the multi-signal approach (Section 6).

P9. Use Cross-Model Verification for Diversity Gain. *Grounding: Proposition 7.2, Littlewood-Strigini.* A weak independent detector ($d = 0.3$, $\rho = 0$) can outperform a strong correlated detector ($d = 0.7$, $\rho = 0.8$) in marginal DRE improvement. When using LLM-as-Judge (Layer 3), use a judge from a different model family than the code producer.

10. Engaging Contrarian Positions

The framework's formal results invite strong objections. Rather than constructing strawmen, we steelman five contrarian positions and evaluate what the evidence supports.

10.1. "Slop Is Acceptable"

The steelmanned argument. Not all code needs to be high-quality. Prototypes, internal tools, and throwaway scripts have low failure costs. Investing in accretion prevention for code that will be discarded is waste.

Response. Partially validated. Theorem 7.3 confirms that the optimal quality investment q^* is less than 1 for low-stakes code. The Accretion Category is primarily dangerous for long-lived production codebases where the compound degradation effect has time to accumulate (the $\binom{n}{2}$ cross-change term in Proposition 3.1). For throwaway code, the appropriate response is to accept accretion and ensure the code is actually thrown away rather than migrating into production. The danger is that "prototype-quality" AI code silently becomes production code through institutional inertia.

10.2. "This Is Just Technical Debt Rebranded"

The steelmanned argument. Cunningham's technical debt metaphor already captures "individually defensible, collectively harmful" code. The Accretion Category adds unnecessary jargon.

Response. The overlap is real but the distinctions matter. Technical debt as defined by Cunningham (Cunningham, 1992) involves *deliberate* shortcuts with *intentional* plans for repayment. Accretion involves *unintentional* superfluity with *no awareness* that debt is being incurred. More concretely: (i) the generative mechanism differs (statistical completion versus intentional choice); (ii) the decidability profile differs (individual accretion is formally undecidable, while design debt is typically recognizable by an expert); (iii) the accumulation dynamics differ (accretion exhibits the compound degradation quadratic, while intentional debt accumulates linearly because a developer who takes a shortcut is at least aware of it). These distinctions have practical consequences: the remediation strategy for technical debt is "pay it back" (refactor the known shortcuts), while the remediation strategy for accretion is "detect it exists" (the hard problem is identification, not repair).

10.3. "The 18 Types Are Wrong"

The steelmanned argument. The specific taxonomy of 18 types is arbitrary. Perhaps the 3-factor structure (F1, F2, F3) is more fundamental, and the 18 types are merely a convenience.

Response. Possibly correct. The 18 types were derived from practitioner experience and iterative refinement, not from principled statistical analysis. Confirmatory factor analysis (CFA) on a labeled dataset of slop instances could discover a different number of latent categories. The 3-factor structure (Context Blindness, Completion Bias, Training Distribution Leakage) is proposed as a generative model, not validated as one. If the factors are not orthogonal (as we conjecture but have not tested), the taxonomy may conflate types that share a common generative root. We consider the 3-factor structure more likely to survive empirical scrutiny than the specific 18-type enumeration.

10.4. "LLM-as-Judge Is Unreliable"

The steelmanned argument. Using one LLM to judge another's output introduces correlated errors (shared training data, shared biases, shared blind spots). Judge reliability is itself uncertain.

Response. Correct as stated; the FKG inequality (Proposition 7.2) formalizes exactly this concern. Same-family judge-producer pairs have correlated failures, and the independence assumption underestimates escape probability. The remedy is cross-model verification: use a judge from a different model family (e.g., Claude judging GPT output, or vice versa). Even at equal detection rates, the diversity gain Δ_{div} is positive whenever $\rho_{AB} < \rho_{AA}$. The empirical question is the magnitude of the diversity gain. Littlewood and Strigini (Littlewood and Strigini, 1993) showed that diverse techniques can achieve effectiveness greater than the independence assumption predicts. The practical recommendation is: never use the same model to both produce and judge code; always use cross-model or cross-technique verification.

10.5. "AI Quality Is Good Enough"

The steelmanned argument. GitHub's RCT found no statistically significant quality difference between AI-assisted and unassisted code (GitHub, 2024). The quality concern is overblown.

Response. The contradiction between the GitHub RCT (short-term, no quality difference) and GitClear's longitudinal data (8× duplication, refactoring collapse) is precisely what the Accretion Category explains. Short-term studies measure point-in-time correctness (property i of the accretion definition). Longitudinal studies measure cumulative coupling growth (property iv). Accretion effects are invisible to short-term studies because each individual change is correct and defensible; they become visible only in aggregate, over time. The GitHub RCT and the GitClear analysis are both correct; they measure different quantities. The Accretion Category provides the theoretical framework for understanding why they diverge.

11. Limitations and Future Work

11.1. Critical Gaps

No labeled dataset. The paper's most significant gap is the absence of a labeled dataset of accretion defects. The formal definitions, taxonomy, and detection methods are proposed but unvalidated. Without ground-truth labels, we cannot measure precision, recall, or F1 for any of the proposed detectors. Creating such a dataset requires expert annotators who can identify superfluous code (property ii), which itself requires understanding the minimal sufficient change, an undecidable problem in general. We propose labeling by expert consensus (3+ senior engineers independently classify changes) as a practical approximation.

Inter-rater reliability. The detectability classification (D/PD/U) and the assignment of slop types to classes have not been validated through inter-rater reliability studies. The κ estimates cited (≈ 0.6 for PD, < 0.4 for U) are extrapolated from general code review literature, not measured for the specific slop types defined here.

Orthogonality of generative factors. The approximate orthogonality of F1, F2, and F3 (Proposition 4.1) is asserted but not tested. Confirmatory factor analysis requires a labeled dataset annotated with both slop type and generative factor, which does not exist.

Aggregate detectors not evaluated. The four aggregate detection signals (Section 6) are proposed but not evaluated on real codebases. Threshold selection, false positive rates, and detection latency

are unknown.

11.2. Verification Roadmap

We propose the following falsification criteria for the paper's key claims:

Table 4 | Verification roadmap with falsification criteria.

Claim	Status	Falsification Criterion
Compound degradation is superlinear	Engineering Hypothesis	$\Gamma(C)$ grows linearly (not quadratically) in controlled experiments with no refactoring
Refactoring ratio equilibrium at $r \geq 0.20$	Empirical extrapolation	Codebases with $r < 0.15$ show no increase in coupling complexity over 12 months
3-factor structure	Engineering Hypothesis	CFA on labeled dataset finds fewer or more factors, or factors are not approximately orthogonal
Detection ceiling bounds	Mathematical Fact	N/A (information-theoretic bound); falsified only if the bound is incorrectly derived
Cross-model diversity gain	Theoretical prediction	Same-family judges achieve equal or better DRE than cross-family judges on matched evaluation sets
Aggregate detection is feasible	Proposed, untested	Multi-signal approach produces $>$ 30% false positive rate on codebases without known accretion

12. Related Work

12.1. Technical Debt Literature

Cunningham's original metaphor (Cunningham, 1992) and Kruchten et al.'s taxonomy (Kruchten et al., 2012) are the foundation. Brown et al. (Brown et al., 2010) provide a systematic treatment; Li et al. (Li et al., 2015) catalogue 94 primary studies. Our contribution extends this literature by identifying a defect class arising from a generative mechanism (statistical completion) that did not exist when these taxonomies were developed. Fowler's code smells (Fowler, 1999) describe many of the structural symptoms that accretion produces (duplicated code, speculative generality, shotgun surgery), but treat them as the result of human design choices rather than statistical generation.

12.2. Defect Taxonomy Literature

IEEE 1044 (IEEE, 2009), ODC (Chillarege et al., 1992), Beizer (Beizer, 1990), and CWE (MITRE, 2024) provide comprehensive defect classification frameworks. None models the "correct but superfluous" combination that defines accretion. Boehm and Basili's "Top 10" (Boehm and Basili, 2001) and Shull et al. (Shull et al., 2002) provide empirical grounding for defect rates and remediation

costs; our cost-of-quality recalibration (Section 7) extends their results to the AI code generation setting.

12.3. AI Code Quality Measurement

CodeRabbit (CodeRabbit, 2025), GitClear (GitClear, 2025), Qodo (Qodo, 2025), and SonarSource (SonarSource, 2026) provide the empirical foundation. GitClear's longitudinal analysis (211M changed lines) is the most comprehensive dataset; its findings (8× duplication, refactoring collapse) directly motivate the Accretion Category. DORA 2025 (Google Cloud, 2025) and Forsgren et al. (Forsgren et al., 2018) provide the organizational-level metrics showing the productivity-quality paradox.

12.4. Software Reliability and Diversity

Littlewood and Strigini (Littlewood and Strigini, 1993, 2000) established that independently developed software versions do not fail independently, the theoretical foundation for our FKG inequality result and the diversity gain analysis. Capers Jones (Jones, 2012) established the 95% DRE threshold for three diverse techniques. Musa (Musa et al., 1987) and Fenton and Neil (Fenton and Pfleeger, 1999) provide the reliability modeling framework that underpins the DRE cascade analysis.

12.5. Software Evolution

Lehman's laws of software evolution (Lehman, 1980) predict that software complexity increases unless work is done to reduce it (the "increasing complexity" law). The Accretion Category provides a specific mechanism by which this law is accelerated under AI code generation: statistical completion systematically favors addition over refactoring, violating the equilibrium condition (Corollary 3.1). Hassan's change entropy (Hassan, 2009) predicts faults from change scattering; our coupling complexity (Definition 3.2) shifts from measuring change scattering to measuring the coupling structure that necessitates it. Hindle et al.'s naturalness of software (Hindle et al., 2012) provides the theoretical basis for why LLMs generate plausible-looking code: software is statistically regular, and LLMs exploit this regularity. The same regularity that makes generation effective also makes accretion likely, because the "most probable" continuation includes patterns that are common in the training data but unnecessary for the specific task.

12.6. Information Theory in Software Engineering

Tishby et al.'s information bottleneck method (Tishby et al., 2000) provides an alternative formalization of the accretion problem: the context window acts as an information bottleneck between the full codebase and the generated code. When the bottleneck is too narrow (context deficit, F1), the agent cannot distinguish between necessary and unnecessary code patterns, and defaults to the most probable completion. Shannon's foundational work (Shannon, 1948) and Cover and Thomas (Cover and Thomas, 2006) provide the information-theoretic tools used in the Detection Ceiling theorem.

13. Conclusion

The Accretion Category identifies a novel defect class that is invisible to existing taxonomies: code that is correct, superfluous, individually defensible, and collectively harmful. This defect class arises specifically from the generative mechanism of statistical pattern completion, and it explains the paradox of individually correct changes producing collectively degrading codebases.

Three key results emerge. First, individual accretion detection is undecidable (Theorem 5.1), which means no fully automated per-change detector can eliminate it. Second, aggregate accretion is detectable via statistical process control on coupling complexity, refactoring ratio, clone frequency, and dependency fan-out (Section 6). Third, the compound degradation proposition (Proposition 3.1) predicts superlinear coupling growth when refactoring does not keep pace with addition, a prediction consistent with GitClear's observed 8× duplication increase and refactoring collapse.

The practical implication is a shift in quality strategy: from detecting individual defects to monitoring aggregate codebase health, from point-in-time checks to longitudinal trend analysis, and from same-model self-verification to cross-model diverse verification. The nine design principles (Section 9) provide actionable guidance; the most urgent is maintaining the refactoring ratio above 20% (P1), which the data shows is currently violated by approximately 2.5×.

The paper's primary limitation is the absence of empirical validation. The formal definitions, taxonomy, and detection methods are proposed, not tested. The verification roadmap (Table 4) identifies the specific experiments needed. Until those experiments are conducted, the Accretion Category remains a well-motivated hypothesis rather than an established result.

References

B. Beizer. *Software Testing Techniques*. Van Nostrand Reinhold, 2nd edition, 1990.

B. Boehm and V. R. Basili. Software defect reduction top 10 list. *Computer*, 34(1):135–137, 2001.

B. W. Boehm. *Software Engineering Economics*. Prentice-Hall, 1981.

N. Brown, Y. Cai, Y. Guo, R. Kazman, M. Kim, P. Kruchten, E. Lim, A. MacCormack, R. Nord, I. Ozkaya, R. Sangwan, C. Seaman, K. Sullivan, and N. Zazworka. Managing technical debt in software-reliant systems. pages 47–52, 2010.

R. Chillarege, I. S. Bhandari, J. K. Chaar, M. J. Halliday, D. S. Moebus, B. K. Ray, and M.-Y. Wong. Orthogonal defect classification: A concept for in-process measurements. *IEEE Transactions on Software Engineering*, 18(11):943–956, 1992.

CodeRabbit. AI code review quality analysis, 2025.

T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

W. Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, pages 29–30, 1992.

N. E. Fenton and S. L. Pfleeger. *Software Metrics: A Rigorous and Practical Approach*. PWS Publishing, 2nd edition, 1999.

N. Forsgren, J. Humble, and G. Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

M. Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.

C. Gao, J. Zeng, M. R. Lyu, and I. King. Online app review analysis for identifying emerging issues. *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, 2017.

GitClear. AI copilot code quality research 2025. https://www.gitclear.com/ai_assistant_code_quality_2025_research, 2025.

GitHub. Research: Quantifying GitHub Copilot's impact in the enterprise, 2024.

Google Cloud. 2025 DORA report. https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report, 2025.

A. E. Hassan. Predicting faults using the complexity of code changes. *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pages 78–88, 2009.

A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu. On the naturalness of software. *Communications of the ACM*, 59(5):122–131, 2012.

IEEE. IEEE standard classification for software anomalies. Technical Report IEEE Std 1044-2009, IEEE Computer Society, 2009.

C. Jones. *The Economics of Software Quality*. Addison-Wesley, 2012.

J. M. Juran and A. B. Godfrey. *Juran's Quality Handbook*. McGraw-Hill, 5th edition, 1999.

P. Kruchten, R. L. Nord, and I. Ozkaya. Technical debt: From metaphor to theory and practice. *IEEE Software*, 29(6):18–21, 2012.

M. M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9): 1060–1076, 1980.

Z. Li, P. Avgeriou, and P. Liang. A systematic mapping study on technical debt and its management. *Journal of Systems and Software*, 101:193–220, 2015.

B. Littlewood and L. Strigini. Validation of ultrahigh dependability for software-based systems. *Communications of the ACM*, 36(11):69–80, 1993.

B. Littlewood and L. Strigini. Software reliability and dependability: A roadmap. *Proc. Conference on The Future of Software Engineering*, pages 175–188, 2000.

METR. Reward hacking in AI agent benchmarks, 2025.

METR. Do SWE-bench verified solutions actually get merged?, 2026.

MITRE. Common weakness enumeration (CWE). https://cwe.mitre.org, 2024.

J. D. Musa, A. Iannino, and K. Okumoto. *Software Reliability: Measurement, Prediction, Application*. McGraw-Hill, 1987.

G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions. *Mathematical Programming*, 14:265–294, 1978.

Qodo. Qodo: AI-powered code quality platform. https://www.qodo.ai, 2025.

H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953.

C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3): 379–423, 1948.

F. Shull, V. Basili, B. Boehm, A. W. Brown, P. Costa, M. Lindvall, D. Port, I. Rus, R. Tesoriero, and M. Zelkowitz. What we have learned about fighting defects. pages 249–258, 2002.

SonarSource. The architecture gap: Why your code becomes hard to change. https://www.sonarsource.com/blog/the-architecture-gap-why-your-code-becomes-hard-to-change/, 2026.

Spotify Engineering. Spotify engineering practices for AI-assisted development. Spotify Engineering Blog, 2025.

N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. *Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing*, pages 368–377, 2000.

Wang et al. AgentSpec: Specification and verification of AI agent behavior, 2026.